

Projet Chatbot Poulpes

Knowledge Graph Pipeline

Construction · Alignement · Raisonnement & KGE · RAG

Bilal Gougis & Chadi Al Kerdi

Cours : Web Mining & Sémantique

Mars 2026

1. Acquisition de Données & Extraction d'Information	3
1.1 Un choix de passionné	3
1.2 L'épreuve du Crawler	3
1.3 Extraction NLP	3
1.4 Cas d'Ambiguïté	3
2. Construction & Expansion de la Base de Connaissances	5
2.1 Modélisation RDF	5
2.2 Entity Linking	5
2.3 Alignement des Prédicats	5
2.4 Stratégie d'Expansion.....	5
2.5 Statistiques Finales	5
3. Raisonnement SWRL	7
4. Knowledge Graph Embeddings.....	8
4.1 Préparation des Données	8
4.2 Modèles & Configuration	8
4.3 Résultats	8
4.4 Analyse Head vs Tail.....	8
4.5 Sensibilité à la taille	8
4.6 Voisins Proches & PCA	9
5. RAG sur RDF/SPARQL	10
5.1 Résumé de Schéma	10
5.2 Génération SPARQL	10
5.3 Self-Repair	10
5.4 Évaluation	10
5.5 Interface	10
6. Réflexion Critique.....	11
6.1 La donnée est reine.....	11
6.2 Le fossé NLP/KGE	11
6.3 Rule-based vs Embedding.....	11
6.4 Améliorations possibles.....	11

1. Acquisition de Données & Extraction d'Information

1.1 Un choix de passionné

Le choix de ce domaine ne doit rien au hasard. En tant que pêcheur de poulpes, j'ai (Bilal) une vraie fascination pour l'intelligence et la biologie de ces céphalopodes. Avec plus de 300 espèces décrites et des sources dispo en français comme en anglais, c'était un terrain de jeu parfait pour du Web Sémantique.

On a ciblé 38 pages web depuis 4 sources principales :

Source	Pages	Mots	Type
Wikipedia FR	16	20 402	Encyclopédie (API)
MarineSpecies.org	5	5 579	Base taxonomique
URLs spécialisées	16	27 083	Reporterre, Larousse, DORIS...
GuideDesEspèces.org	1	1 452	Guide de pêche durable
Total	38	55 516	

1.2 L'épreuve du Crawler

Le plus gros défi au début c'était la qualité des données. Notre crawler ramenait des trucs qui n'avaient rien à voir avec les poulpes, genre des pages sur des films ou de la géopolitique (on en reparle en section 6 parce que c'était vraiment le gros problème du projet).

Du coup on a dû sélectionner et vérifier manuellement nos 38 sources de confiance. Le crawler utilise l'API Wikipedia pour les articles encyclopédiques et BeautifulSoup pour les autres sites. Chaque page est vérifiée contre le robots.txt avant le téléchargement, avec un délai d'1 seconde entre chaque requête pour respecter les serveurs.

Les pages de moins de 150 mots sont rejetées, le contenu est nettoyé (on vire les balises nav, footer, script, style) et dédoublé par hash MD5. Tout est stocké en JSONL.

1.3 Extraction NLP

L'extraction utilise spaCy avec le modèle fr_core_news_md. On extrait deux types d'info :

NER : 2 646 entités uniques (1 043 MISC, 716 PER, 612 LOC, 258 ORG, et 17 SPECIES qu'on a ajoutées nous-mêmes via un dictionnaire spécifique au domaine parce que spaCy ne connaît pas les noms d'espèces).

Relations : 503 triplets sujet-verbe-objet extraits par analyse des dépendances syntaxiques. Les verbes les plus fréquents : posséder (16), faire (13), utiliser (12), contenir (11).

1.4 Cas d'Ambiguïté

On a repéré 3 cas intéressants :

1. Octopus vulgaris : spaCy le classifie parfois comme PER (personne) parce que le format binominal ressemble à un nom propre. Notre dictionnaire ESPECES corrige ça.

2. Manche : classifié LOC par spaCy, ce qui est correct pour le département, mais dans notre contexte maritime c'est la Manche (la mer). L'entity linking avec Wikidata résout l'ambiguïté.

3. Chromatophore : classé MISC alors que c'est un organe biologique précis. La catégorie MISC de spaCy c'est un peu le fourre-tout, faudrait un post-traitement pour ça.

2. Construction & Expansion de la Base de Connaissances

2.1 Modélisation RDF

L'ontologie définit 9 classes (Species, Researcher, Institution, Location, Behavior, BodyPart, Habitat, Predator, Prey) et 4 propriétés d'objet (habitatOf, preyOf, studiedBy, locatedIn). Les namespaces : oct: pour l'ontologie, ent: pour les entités.

La KB initiale fait environ 8 800 triplets. Petit détail important : les relations privées sont de type entité → littéral texte (genre ent:pieuvre oct:masquer "sa fuite"). C'est une limitation pour le KGE qu'on détaille en section 6.

2.2 Entity Linking

On a aligné nos entités avec Wikidata via l'API wbsearchentities. Résultat : 397 entités liées sur 500 candidates (79%), confiance moyenne de 0.948. Par exemple ent:Octopus_vulgaris → wd:Q190782, ent:mer_Méditerranée → wd:Q4918. Les entités non liées c'est souvent des expressions trop complexes ou spécifiques.

2.3 Alignement des Prédicats

5 prédicats privés sur 30 ont été alignés manuellement : posséder → P527 (has part), utiliser → P366 (use), trouver → P131 (located in), consommer → P1034 (main food source), produire → P1056 (product produced). Les 25 restants (faire, voir, agrandir...) n'ont pas d'équivalent direct dans Wikidata, c'est normal ces verbes sont trop génériques.

2.4 Stratégie d'Expansion

L'expansion repose sur des entités ancres biologiques vérifiées : Q40152 (Octopoda), Q26899 (Cephalopoda), Q131257 (Octopus), Q190782 (Octopus vulgaris), Q1266322 (Enteroctopus dofleini), Q1476486 (Hapalochlaena), etc.

Point critique (le plus gros bug du projet) : notre première version utilisait des QIDs complètement faux. Q1077 c'était un FILM appelé "Octopus", Q192882 c'était le gouvernorat de Najaf en Irak, Q76125 une personne. Du coup l'expansion 2-hop ramenait des milliers de triplets sur la diplomatie, le cinéma et la linguistique. On a passé un bon moment à comprendre pourquoi nos embeddings apprenaient la géopolitique au lieu de la biologie marine. Après correction, l'expansion 2-hop est ciblée uniquement sur les taxons et un nettoyage post-expansion vire tous les prédicats hors-domaine.

2.5 Statistiques Finales

Métrique	Valeur
Triplets totaux	~20 600
Entités privées (ent:)	~8 600
Triplets Wikidata	~11 800
P171 (parent taxon)	101
P105 (taxon rank)	81
P1672 (preys on)	19

P150/P530/P2936 (bruit)	0 (nettoyé)
-------------------------	-------------

3. Raisonnement SWRL

On a implémenté une règle SWRL avec OWLReady2 pour montrer le raisonnement déductif :

Règle : $Species(?s) \wedge venom_level(?s, ?v) \wedge greaterThan(?v, 0) \rightarrow DangerousSpecies(?s)$

L'idée c'est simple : si une espèce a du venin, elle est classée comme dangereuse automatiquement. Appliqué à Hapalochlaena (venom_level=5, la pieuvre à anneaux bleus qui est mortelle pour l'humain) et Octopus vulgaris (venom_level=0, inoffensif), seul Hapalochlaena devrait être classifié DangerousSpecies.

Le mur Java : on s'est heurté à un problème avec le raisonneur Pellet qui demandait Java 21+ alors qu'on tournait sur Java 8. On a pas réussi à le faire marcher directement mais les résultats attendus sont documentés et la règle SWRL est bien définie dans le code.

4. Knowledge Graph Embeddings

C'est la partie qui nous a donné le plus de fil à retordre. Obtenir des scores décents a été un vrai combat.

4.1 Préparation des Données

On nettoie la KB en virant les littéraux, les métadonnées (rdf:type, rdfs:label, owl:sameAs, prov:wasDerivedFrom), les URIs de fichiers, et les identifiants externes Wikidata (P18, P214, P646...). Les relations avec moins de 3 occurrences sont aussi filtrées pour éviter le bruit.

Split 80/10/10 : ~8 200 triplets d'entraînement, ~1 000 de validation, ~1 000 de test.

4.2 Modèles & Configuration

On a entraîné 3 modèles avec les mêmes hyperparams pour une comparaison équitable : embedding_dim=200, Adam (lr=5e-4), bernoulli negative sampling (10 négatifs par positif), early stopping sur Hits@10 (patience=10).

TransE : MarginRankingLoss (margin=2.0). C'est un modèle translationnel, l'idée c'est que $h + r \approx t$.

DistMult : MarginRankingLoss (margin=2.0). Modèle bilinéaire : score = $h^T \text{diag}(r) t$.

Complex : SoftplusLoss avec régularisation L2 (1e-5). Il utilise des embeddings complexes pour gérer les relations asymétriques. On a dû changer la loss parce que MarginRanking marchait pas du tout avec les nombres complexes.

4.3 Résultats

Modèle	MRR	Hits@1	Hits@3	Hits@10
TransE	0.19	0.07	0.25	0.40
DistMult	0.34	0.24	0.38	0.52
Complex	0.00	0.00	0.00	0.00

DistMult est clairement le meilleur (MRR=0.34, Hits@10=0.52). On est particulièrement contents d'avoir dépassé le seuil de 0.20 pour le Hits@1 après tant d'itérations sur la donnée. TransE fait des performances honorables. Complex a pas convergé du tout sur notre KB de ~10k triplets, probablement pas assez de données pour les embeddings complexes.

4.4 Analyse Head vs Tail

DistMult prédit mieux les tails (MRR tail=0.33 vs head=0.29). C'est cohérent avec la structure taxonomique : prédire «Octopus → P171 → ?» (tail) c'est plus facile parce qu'il y a moins de parents que d'enfants dans un arbre taxonomique.

4.5 Sensibilité à la taille

L'analyse 20k/50k/Full donne exactement les mêmes résultats parce que notre KB fait que ~10k triplets, donc les 3 sous-échantillons contiennent les mêmes données. C'est une limitation mais ça confirme ce que dit l'énoncé : en dessous de 10k triplets on peut pas vraiment observer de différence.

4.6 Voisins Proches & PCA

La visualisation PCA montre principalement des entités de type Location (océans, mers). Les espèces et taxons sont sous-représentés dans le train parce que les relations privées (oct:chasser, oct:vivre) sont de type entité → littéral, incompatibles avec le KGE. C'est le même problème qu'on décrit en section 6.

5. RAG sur RDF/SPARQL

Pour l'interface de discussion on a choisi Llama 3.2 1B via Ollama.

Pourquoi ce modèle ? On a testé des versions plus petites (gemma:2b et autres) mais elles étaient incapables de générer du SPARQL cohérent. Llama 3.2 1B c'est le compromis : il met environ 30s à répondre sur CPU mais c'est le seul qui offrait un équilibre acceptable entre compréhension du schéma et hallucinations limitées.

5.1 Résumé de Schéma

Le résumé injecté dans les prompts contient les préfixes RDF, les classes et prédicats avec leur fréquence, les labels des entités clés, et des exemples de triplets. La taille est adaptée au modèle (~1500 caractères pour Llama 3.2 1B).

5.2 Génération SPARQL

Le pipeline utilise une approche hybride assez maline :

Templates SPARQL : pour les patterns de questions courants (habitat, taxonomie, espèces, propriétés). Un détecteur de pattern analyse la question et sélectionne le template. C'est beaucoup plus fiable que de laisser le LLM tout générer.

Génération LLM : pour les questions atypiques seulement. La température est à 0.1 pour un output déterministe.

5.3 Self-Repair

Si le SPARQL échoue, le LLM reçoit l'erreur et le schéma pour générer une requête corrigée. En dernier recours un fallback intelligent cherche les propriétés de l'entité mentionnée dans la question. On tombe jamais sur une erreur système.

5.4 Évaluation

8 questions d'évaluation comparent le baseline (LLM seul) au SPARQL-RAG. Le baseline produit des réponses souvent génériques ou hallucinées, alors que le RAG retourne des données ancrées dans le graphe. Une étape de synthèse reformule les résultats SPARQL bruts en français.

5.5 Interface

L'interface Gradio permet de poser des questions, choisir le modèle Ollama, voir la requête SPARQL générée, et comparer baseline vs RAG côte à côte. On a aussi une démo CLI pour tester rapidement.

6. Réflexion Critique

Ce projet nous a montré que la puissance de l'IA ne remplace pas la qualité de la donnée source. C'est la leçon principale.

6.1 La donnée est reine

Passer d'un MRR de 0.12 à 0.34 c'est pas une question d'algorithme. C'est une question de nettoyage des QIDs Wikidata. On a perdu des heures à essayer d'optimiser les hyperparams de TransE alors que le vrai problème c'était qu'on entraînait le modèle sur de la géopolitique irakienne au lieu de la biologie marine. Une fois les QIDs corrigés, le MRR a littéralement doublé sans changer une seule ligne de code d'entraînement.

6.2 Le fossé NLP/KGE

Les relations extraites par spaCy sont de type entité → littéral texte (pieuvre → oct:masquer → "sa fuite"). Le KGE ne marche qu'avec des triplets entité → entité. Du coup les 498 relations privées ne contribuent pas aux embeddings. C'est frustrant mais c'est une limitation structurelle du pipeline NLP → KGE.

6.3 Rule-based vs Embedding

Le raisonnement SWRL c'est explicite, interprétable et garanti correct (Species + venimeux → DangerousSpecies). Les embeddings KGE c'est implicite, ça apprend automatiquement des patterns mais c'est dur à interpréter et ça échoue sur les petites KB. Les deux approches sont complémentaires : SWRL pour les inférences précises, KGE pour la découverte de liens latents.

6.4 Améliorations possibles

- 1. Relations entité→entité** : remplacer les littéraux par des entités (oct:masquer "sa fuite" → oct:uses ent:ink_cloud). Faudrait un post-traitement NLP plus poussé.
- 2. Plus de sources bio** : intégrer DBpedia, NCBI Taxonomy ou GBIF pour enrichir les données taxonomiques et avoir une KB assez grande pour que la sensibilité à la taille marche.
- 3. GPU** : Llama 3.2 1B en CPU c'est lent (~30s/réponse). Avec un GPU la latence tomberait à 2-3 secondes et on pourrait utiliser un modèle plus gros (Mistral 7B) pour du SPARQL de bien meilleure qualité.
- 4. Évaluation automatique** : implémenter des métriques automatiques (F1 sur les entités retournées) au lieu de l'évaluation manuelle.